

Contents

□ Topological Sort — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	14
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	17
SECTION 19 — Advanced Concepts	17
SECTION 20 — Senior Engineer Insights	18
SECTION 21 — Cheat Sheet	18
SECTION 22 — Revision Notes (5-Minute Review)	18
SECTION 23 — Practice Roadmap	18
SECTION 24 — Memory Tricks	19
SECTION 25 — Final Summary	19

□ Topological Sort — Complete Interview Handbook

Pattern #20 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Graph section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Topological Sort produces a **linear ordering of nodes in a Directed Acyclic Graph (DAG)** such that for every directed edge $u \rightarrow v$, u appears **before** v in the ordering. It's the formalization of "do things in an order that respects all dependency constraints" — and it's only possible when the graph has no cycles (a cycle would create a contradictory "must come before itself" requirement).

1.2 Why Was This Pattern Invented?

Many real-world processes have prerequisite/dependency constraints (course prerequisites, build-system dependencies, task scheduling) — you need a valid execution order satisfying all "X before Y" rules simultaneously. Topological Sort formalizes finding such an order (or detecting that no valid order exists, because of a circular dependency) in a single $O(V+E)$ pass.

1.3 Real Intuition Behind The Pattern

Imagine **getting dressed** — you must put on socks before shoes, and underwear before pants, but the shirt can go on anytime relative to the pants. Topological Sort finds *some* valid sequence honoring all these "before" constraints simultaneously — there may be multiple valid answers (shirt could go first or last), but every valid answer respects every constraint.

1.4 Mental Model

"What has zero remaining unmet prerequisites right now?" — repeatedly find and remove nodes with no incoming edges (or, via DFS, finish deep dependency chains first then reverse the finish order), since those are always safe to schedule next.

1.5 Visual Explanation

Course prerequisites ($u \rightarrow v$ means u must be taken before v):

```
1 → 2
1 → 3
2 → 4
3 → 4
```

In-degree: 1:0, 2:1, 3:1, 4:2

Kahn's Algorithm (BFS-based):

Queue starts with in-degree-0 nodes: [1]

Process 1: decrement in-degree of 2,3 → 2:0, 3:0 → enqueue both → queue=[2,3]

Process 2: decrement in-degree of 4 → 4:1 → queue=[3]

Process 3: decrement in-degree of 4 → 4:0 → enqueue → queue=[4]

Process 4: no outgoing edges → queue=[]

Topological order: 1, 2, 3, 4 (or 1, 3, 2, 4 - both valid)

1.6 Simple Analogy

Topological Sort is like **planning a multi-course meal prep** — you can't plate the dessert before baking it, and you can't bake it before mixing the batter, but you're free to chop vegetables for the salad at any point that doesn't conflict with other steps; topological sort finds *a* valid step-by-step plan honoring every "must happen before" rule.

1.7 When Should I Immediately Think About Using This Pattern?

- “Course schedule,” “task order with prerequisites,” “build order.”
 - “Can all tasks be completed given dependency constraints?” (cycle detection is a byproduct).
 - Any problem framed as “**X depends on Y**” or “**X must happen before Y.**”
 - “Alien dictionary” style problems (inferring an ordering from partial pairwise constraints).
-

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“prerequisites,” “course schedule”	Direct signal
“build order,” “task order”	Direct signal
“must happen before”	Direct signal
“alien dictionary”	Order inference → topological sort
“can finish all tasks”	Cycle detection via failed topological sort

2.2 Hidden Hints

Any problem describing pairwise “A before B” constraints (even if not phrased as a graph) implies building a directed graph and applying topological sort — the “alien dictionary” problem is the classic example of this disguised framing.

2.3 Interview Clues

Interviewer describes dependencies or prerequisites explicitly, or the problem’s output is described as “a valid order” (implying possibly multiple correct answers) rather than “the order” (implying uniqueness).

2.4 Common Trick Words

“Prerequisite,” “depends on,” “must complete before,” “build,” “schedule” — all imply a directed dependency graph requiring topological ordering.

2.5 What Interviewers Expect

Correct recognition that a valid topological order **only exists if the graph is a DAG** (no cycles) — and that failing to produce a complete ordering (fewer nodes processed than total) is exactly the signal that a cycle exists. Also expected: familiarity with both Kahn’s algorithm (BFS, in-degree based) and DFS-based topological sort (postorder + reverse).

2.6 When NOT To Use This Pattern

- The graph is **undirected** — topological order is only meaningful for directed graphs with dependency semantics.
 - The graph **has cycles** and a valid full ordering isn’t possible — the correct answer is to detect and report this (often via Kahn’s algorithm processing fewer than all nodes, or DFS-based cycle detection).
 - You need the **shortest** or **optimal** ordering by some cost metric, not just *any* valid dependency-respecting order — that likely requires a different algorithm (e.g., critical path method / longest path in a DAG) layered on top of the topological structure.
-

SECTION 3 — Decision Framework

Is the graph DIRECTED, with edges representing "must come before" constraints?

Yes

Do you need to know IF a valid complete ordering exists (cycle check) or PRODUCE one?

Both → USE TOPOLOGICAL SORT (Kahn's BFS or DFS-based)

Do you prefer an ITERATIVE, in-degree-counting approach (also naturally detects cycles by incomplete processing) or a RECURSIVE, postorder-based approach?

Yes → USE KAHN'S ALGORITHM (BFS-based)

No - prefer a RECURSIVE, postorder-based approach?

Yes → USE DFS-BASED TOPOLOGICAL SORT (postorder, then reverse)

Do you need the graph's LONGEST PATH or a COST-OPTIMAL ordering (not just any valid order)?

Yes → Layer a DP computation ON TOP of the topological order (process nodes in topological order, updating longest-path/cost values as you go)

Why: Both Kahn's algorithm and DFS-based topological sort correctly produce a valid ordering (or detect impossibility) in $O(V+E)$ — the choice between them is largely stylistic, though Kahn's algorithm's in-degree counting naturally generalizes to "process nodes in dependency-respecting order while accumulating some additional value" (like longest path in a DAG), a common follow-up extension.

SECTION 4 — Why This Pattern Works

Mathematical/Logical (Kahn's Algorithm): A node with in-degree 0 has no unmet prerequisites, so it's always safe to schedule it next. Removing it (and decrementing the in-degree of its neighbors) can only ever create *more* in-degree-0 nodes, never remove the possibility of completing valid nodes — this greedy "always process an available node" approach is provably correct because a DAG is guaranteed to always have at least one in-degree-0 node at every stage (a cycle-free directed graph cannot have all nodes with in-degree ≥ 1 , since that would imply an infinite descending chain, impossible in a finite graph, or a cycle).

Mathematical/Logical (DFS-based): Performing a postorder DFS traversal and then **reversing** the postorder sequence produces a valid topological order because: a node finishes (postorder) only after all its descendants (things it depends on, in a "u → v means u before v" edge convention where DFS explores from u to v) have finished — so reversing the finish order places dependencies correctly before their dependents.

Correctness Proof (Kahn's): *Invariant:* every node added to the result list has had all its prerequisites already added earlier in the list. *Base case:* initial in-degree-0 nodes have no prerequisites — trivially satisfy the invariant. *Inductive step:* a node only becomes eligible (in-degree reaches 0) after all its prerequisite edges have been "consumed" by already-processed (and thus already-added) predecessor nodes — preserving the invariant. *Termination:* if the graph is a DAG, all n nodes are eventually processed (proven by the "always at least one in-degree-0 node available" argument); if fewer than n nodes are processed when the queue empties, a cycle exists among the unprocessed nodes (each has an unmet, never-resolvable prerequisite). **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Kahn's Algorithm, BFS-based)

1. Compute the in-degree of every node.
2. Initialize a queue with all nodes having in-degree 0.
3. While the queue is non-empty: dequeue a node, add it to the result, and for each of its outgoing neighbors, decrement their in-degree; if any neighbor's in-degree reaches 0, enqueue it.
4. If the result contains all n nodes, it's a valid topological order; otherwise, a cycle exists (not all nodes could be processed).

5.2 General Template — Kahn's Algorithm

```
function topologicalSort(graph, n):
    inDegree = array of zeros, size n
    for u in graph.nodes:
        for v in graph.neighbors(u):
            inDegree[v] += 1

    queue = [node for node in range(n) if inDegree[node] == 0]
    result = []

    while queue is not empty:
        node = queue.dequeue()
        result.append(node)
        for neighbor in graph.neighbors(node):
            inDegree[neighbor] -= 1
            if inDegree[neighbor] == 0:
                queue.enqueue(neighbor)

    if length(result) != n:
        return "CYCLE DETECTED - no valid topological order exists"
    return result
```

5.3 General Template — DFS-Based Topological Sort

```
function topologicalSortDFS(graph, n):
    visited = set()
    postorderStack = []

    function dfs(node):
        visited.add(node)
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                dfs(neighbor)
        postorderStack.push(node)                # add AFTER processing all descendants

    for node in range(n):
        if node not in visited:
            dfs(node)

    return reverse(postorderStack)                # postorder reversed = topological order
```

5.4 Interview Thinking Process

1. "This has directed dependency constraints — I'll model it as a graph and check if it's a DAG via topological sort."
 2. "I'll use Kahn's algorithm: repeatedly process in-degree-0 nodes, since they have no unmet prerequisites."
 3. "If I can't process all nodes (result size < n), that means a cycle exists — no valid order is possible."
 4. "Alternatively, I could use DFS: do a postorder traversal and reverse it, since a node only 'finishes' after everything it depends on has finished."
-

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(V + E)$	$O(V + E)$ for the graph + $O(V)$ for in-degree/visited arrays	Every vertex processed once, every edge examined once (for in-degree computation or DFS traversal)
Average Case	$O(V + E)$	$O(V + E)$	Same regardless of graph structure
Best Case	$O(V + E)$ (must still examine all edges to build in-degrees)	$O(V + E)$	Even a graph with an obvious ordering still needs full in-degree computation
Amortized	$O(V + E)$ total across the single pass	$O(V + E)$	No repeated work

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty graph	No nodes	Return an empty ordering immediately
Single node, no edges	Isolated node	Trivially its own valid topological order
Graph with a cycle	1→2→3→1	Kahn's algorithm processes fewer than n nodes; DFS-based approach would need explicit cycle detection (3-color marking) since simple postorder reversal doesn't inherently detect cycles

Edge Case	Example	Handling
Disconnected DAG components	Multiple independent dependency chains	Both algorithms naturally handle this — Kahn’s processes all in-degree-0 nodes across all components; DFS iterates over all unvisited nodes
Multiple valid topological orders	Diamond-shaped dependency (see §1.5)	Any valid order is acceptable unless the problem specifies additional tie-breaking (e.g., lexicographically smallest, requiring a min-heap instead of a plain queue in Kahn’s algorithm)
Self-loop	Node depends on itself	Immediately a cycle — in-degree never reaches 0 for that node, correctly detected as impossible
Disconnected node with no dependencies at all	Isolated node in a larger graph	Correctly included as an in-degree-0 node processed at some point

Common mistakes: using plain DFS without explicit 3-color cycle detection and assuming postorder-reversal alone detects cycles (it doesn’t — you need separate cycle detection, or rely on Kahn’s algorithm’s incomplete-processing signal instead); forgetting to check whether the result includes all n nodes when using Kahn’s algorithm, silently returning an incomplete/invalid order.

SECTION 8 — Pros & Cons

Advantages: $O(V+E)$ — optimal; naturally detects cycles (Kahn’s algorithm) as a side effect of attempting the sort; directly models real-world dependency/prerequisite scheduling problems. **Disadvantages:** Only applicable to directed graphs; doesn’t produce a *unique* ordering when multiple valid orders exist (may need additional constraints/tie-breaking for problems requiring a specific one, like lexicographically smallest). **Trade-offs:** Kahn’s Algorithm (iterative, BFS-style, naturally extensible to lexicographic tie-breaking via a min-heap instead of a plain queue) vs. DFS-based (recursive, postorder-based, requires separate cycle detection) — Kahn’s is often preferred in interviews for its natural cycle-detection byproduct. **Limitations:** Doesn’t directly solve “shortest/longest path” or “cost-optimal ordering” — those require layering additional DP computation on top of the topological structure. **Inefficient when:** N/A for its exact use case — $O(V+E)$ is optimal for producing a valid ordering or detecting a cycle.

SECTION 9 — Real World Applications

Domain	Application
Build Systems (Make, Bazel, Gradle)	Determining the correct build order for modules with interdependencies
Package Managers (npm, pip, apt)	Dependency resolution order for installing packages with prerequisite libraries

Domain	Application
Compilers	Determining compilation order for source files with <code>#include/import</code> dependencies
Spreadsheet Applications	Determining cell recalculation order when formulas reference other cells
Project Management Tools (Atlassian/Jira)	Task scheduling respecting “blocked by” dependency relationships
CI/CD Pipelines	Determining the execution order of pipeline stages with dependency constraints
Course Registration Systems	Course prerequisite validation and suggested enrollment ordering (the canonical “Course Schedule” problem framing)
Database Systems	Query execution plan step ordering respecting operator dependencies
Distributed Systems	Service startup ordering respecting inter-service dependency requirements
Version Control	Dependency-aware merge/rebase ordering in certain advanced git workflows

SECTION 10 — Interview Strategy

How seniors answer: They immediately model the problem as a directed graph with dependency edges, recognize that a valid ordering only exists if the graph is a DAG, and choose between Kahn’s (iterative, natural cycle detection) and DFS-based (recursive, requires separate cycle detection) with a clear rationale.

How juniors answer: They sometimes attempt a naive greedy approach without recognizing the underlying graph structure, or they use DFS-based postorder without adding explicit cycle detection, incorrectly assuming any DFS traversal handles cycles safely.

Typical follow-ups: “What if a cycle exists — how do you detect it?” “Can you produce the lexicographically smallest valid order?” (Use a min-heap instead of a plain queue in Kahn’s algorithm.) “How would you find the longest path in this DAG?” (Process nodes in topological order, applying a DP relaxation step at each node.)

Optimization questions: “Can you do this without extra space for the in-degree array?” (Not typically — the in-degree computation is fundamental to Kahn’s algorithm’s correctness.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Kahn’s Algorithm (BFS)	In-degree-based iterative processing	Course Schedule II
DFS-Based Topological Sort	Postorder traversal + reversal	Alien Dictionary (DFS variant)
Cycle Detection via Topological Sort	Incomplete Kahn’s processing signals a cycle	Course Schedule
Lexicographically Smallest Topological Order	Min-heap instead of plain queue in Kahn’s algorithm	Course Schedule variants with tie-breaking
Longest Path in a DAG	DP layered on top of topological order	Longest Path with Different Adjacent Characters

Variation	Description	Example
All Valid Topological Orders	Backtracking combined with in-degree tracking	Generating all valid course orderings (custom/interview variant)
Order Inference from Partial Constraints	Build the graph from pairwise comparisons first	Alien Dictionary

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Graph BFS/DFS (general)	No dependency-ordering guarantee, just connectivity/reachability	Need connectivity, not a valid dependency-respecting sequence
Union-Find	Handles undirected connectivity/cycle detection, not directed dependency ordering	Undirected grouping/connectivity, not directed “before” constraints
Dynamic Programming on DAGs	Often layered ON TOP of a topological order for optimal-value computation	Need cost-optimal ordering or longest/shortest path, not just any valid order
Greedy Algorithms	Kahn’s algorithm is itself a greedy approach (always process an available node)	N/A — Topological Sort via Kahn’s IS an application of the Greedy pattern

Comparison Table

Aspect	Kahn’s Algorithm (BFS)	DFS-Based Topological Sort
Style	Iterative, in-degree counting	Recursive, postorder + reverse
Cycle detection	Natural (incomplete processing)	Requires separate 3-color marking
Extensibility	Naturally extends to lex-smallest order (min-heap)	Less naturally extensible

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	N/A (Topological Sort itself rarely appears at pure Easy difficulty)	—
Medium	Basic ordering/cycle detection	Course Schedule, Course Schedule II

Difficulty	Characteristics	Examples
Hard	Order inference from partial constraints, DP layered on topological order	Alien Dictionary, Longest Path with Different Adjacent Characters
Very Hard	Multi-constraint combinations, advanced tie-breaking	Sequence Reconstruction, Parallel Courses III

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Course Schedule	Medium	Amazon, Meta, Google, Microsoft	Cycle detection via topological sort	Foundational cycle detection
2	Course Schedule II	Medium	Amazon, Meta, Google	Producing a valid topological order	Foundational ordering mechanics
3	Alien Dictionary	Hard	Amazon, Meta, Google, Microsoft	Order inference + topological sort	Advanced constraint-graph construction
4	Sequence Reconstruction	Medium	Google, Amazon	Uniqueness checking in topological order	Uniqueness verification
5	Course Schedule III (contrast, greedy/heap)	Hard	Google, Amazon	Contrast: greedy + heap, not pure topological sort	Pattern-boundary awareness
6	Parallel Courses	Medium	Google, Amazon	Kahn's algorithm for level-based scheduling	Level-based topological processing
7	Parallel Courses II	Hard	Google	Advanced constrained topological scheduling with capacity limits	Advanced constrained scheduling
8	Parallel Courses III	Hard	Google, Amazon	DP layered on topological order for minimum completion time	Cross-pattern (Topo Sort + DP)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
9	Minimum Height Trees	Medium	Google, Amazon	BFS-based “peeling” similar in spirit to Kahn’s algorithm	Related leaf-peeling technique
10	Longest Path With Different Adjacent Characters	Hard	Google	DP layered on topological/DFS order	Cross-pattern (Topo Sort + DP)
11	Build a Matrix With Conditions	Hard	Google, Amazon	Dual topological sort (row and column constraints)	Advanced dual-constraint topological sort
12	Find All Possible Recipes from Given Supplies	Medium	Amazon, Google	Topological sort for dependency resolution	Dependency resolution application
13	Sort Items by Groups Respecting Dependencies	Hard	Google, Amazon	Dual-level topological sort (groups and items)	Advanced hierarchical topological sort
14	Loud and Rich	Medium	Google, Amazon	DFS-based topological reasoning with memoization	Cross-pattern (Topo Sort + Memoization)
15	Minimum Number of Semesters to Finish All Courses	Medium/Hard	Google	Bitmask DP layered on topological/dependency structure	Cross-pattern (Topo Sort + Bitmask DP)
16	Strange Printer II (contrast)	Hard	Google	Contrast: uses topological-sort-like elimination reasoning	Related elimination technique
17	Course Schedule IV	Medium	Google, Amazon	Reachability computation using topological order	Reachability via topological processing
18	Evaluate the Bracket Pairs of a String (contrast)	Medium	Amazon	Contrast: hashmap-based, not topological sort	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
19	Determine if String Halves Are Alike (contrast)	Easy	Amazon	Contrast: unrelated, included for boundary practice	Pattern-boundary awareness
20	Find Eventual Safe States	Medium	Google, Amazon	Reverse-graph topological sort (or DFS coloring)	Reverse-edge topological reasoning
21	All Ancestors of a Node in a Directed Acyclic Graph	Medium	Google, Amazon	Topological order + ancestor propagation	Cross-pattern (Topo Sort + Propagation)
22	Shortest Path in Directed Acyclic Graph (custom/interview variant)	Medium	Google	DP layered on topological order for shortest path	Cross-pattern (Topo Sort + DP shortest path)
23	Largest Color Value in a Directed Graph	Hard	Google, Amazon	DP + topological sort for path color counting	Advanced cross-pattern combination
24	Number of Ways to Arrive at Destination (contrast, weighted)	Medium	Amazon, Google	Contrast: needs Dijkstra's, not pure topological sort	Pattern-boundary awareness
25	Reconstruct Itinerary (contrast, Eulerian path)	Hard	Google, Amazon	Contrast: Eulerian path (Hierholzer's algorithm), not topological sort	Pattern-boundary awareness
26	Verify Preorder Serialization of a Binary Tree (contrast)	Medium	Amazon	Contrast: stack-based validation, not topological sort	Pattern-boundary awareness
27	Task Scheduler (contrast, greedy/heap)	Medium	Amazon, Meta, Google	Contrast: greedy + heap scheduling, not topological sort	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
28	Minimum Time to Complete All Tasks (contrast)	Hard	Google	Contrast: greedy interval scheduling, not topological sort	Pattern-boundary awareness
29	Detect Cycles in 2D Grid (contrast, undirected)	Medium	Google, Amazon	Contrast: undirected cycle detection (DFS/Union-Find), not topological sort	Pattern-boundary awareness
30	Design a course-scheduling system (custom/interview variant)	Hard	Google, Amazon (system design-adjacent)	Full applied topological sort system design	Applied system design

SECTION 15 — Common Mistakes

1. Assuming plain DFS-based postorder-reversal automatically handles cycle detection without adding explicit 3-color marking — a cyclic graph would cause infinite recursion or silently incorrect results without it. *Fix:* either use Kahn’s algorithm (whose incomplete processing naturally signals a cycle) or add explicit 3-color DFS cycle detection alongside the postorder approach.
2. Forgetting to check whether Kahn’s algorithm’s result contains all n nodes before declaring success — silently returning an incomplete (invalid) order. *Fix:* always compare `length(result) == n` before returning.
3. Building the dependency graph with edges in the wrong direction (e.g., “A depends on B” should be an edge $B \rightarrow A$, not $A \rightarrow B$, depending on convention) — leading to a reversed or non-sensical ordering. *Fix:* explicitly clarify and consistently apply the edge-direction convention before coding.
4. Using a plain queue in Kahn’s algorithm when the problem requires a specific tie-breaking order (e.g., lexicographically smallest) — a plain FIFO queue doesn’t guarantee this; a min-heap is needed instead. *Fix:* recognize tie-breaking requirements and swap in a min-heap.
5. Forgetting that topological sort only applies to directed graphs, and attempting to apply it to an undirected graph’s structure. *Fix:* always confirm edge directionality is meaningful before applying this pattern.

Why people fail: the core algorithms (Kahn’s, DFS-based) are simple, but candidates often forget the *validation* step (checking Kahn’s result size, or adding cycle detection to DFS) that turns “an ordering” into “a **correct** ordering, or a correctly-detected impossibility” — this validation step is exactly what interviewers probe with cyclic test cases.

SECTION 16 — Optimization Techniques

- **Time:** Already optimal at $O(V+E)$ — no further asymptotic improvement possible for producing a valid order or detecting a cycle.
 - **Space:** Minimal additional space beyond the graph representation itself (in-degree array or visited set, both $O(V)$).
 - **Readability:** Clearly separate “build the graph and in-degree counts,” “process the queue,” and “validate completeness” into distinct, well-commented code sections.
 - **Interview performance:** Explicitly state the cycle-detection mechanism (incomplete Kahn’s processing, or DFS 3-color marking) before coding — this proactively addresses the most common correctness follow-up.
-

SECTION 17 — Multiple Language Templates

Java

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    int[] inDegree = new int[numCourses];
    for (int[] p : prerequisites) {
        graph.get(p[1]).add(p[0]);
        inDegree[p[0]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.offer(i);
    int[] result = new int[numCourses];
    int idx = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        result[idx++] = node;
        for (int neighbor : graph.get(node)) {
            if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
    }
    return idx == numCourses ? result : new int[0];
}
```

JavaScript

```
function findOrder(numCourses, prerequisites) {
    const graph = Array.from({length: numCourses}, () => []);
    const inDegree = new Array(numCourses).fill(0);
    for (const [course, prereq] of prerequisites) {
        graph[prereq].push(course);
        inDegree[course]++;
    }
    const queue = [];
    for (let i = 0; i < numCourses; i++) if (inDegree[i] === 0) queue.push(i);
    const result = [];
    while (queue.length) {
        const node = queue.shift();
        result.push(node);
    }
}
```

```

        for (const neighbor of graph[node]) {
            if (--inDegree[neighbor] === 0) queue.push(neighbor);
        }
    }
    return result.length === numCourses ? result : [];
}

```

PHP

```

function findOrder(int $numCourses, array $prerequisites): array {
    $graph = array_fill(0, $numCourses, []);
    $inDegree = array_fill(0, $numCourses, 0);
    foreach ($prerequisites as [$course, $prereq]) {
        $graph[$prereq][] = $course;
        $inDegree[$course]++;
    }
    $queue = [];
    for ($i = 0; $i < $numCourses; $i++) if ($inDegree[$i] === 0) $queue[] = $i;
    $result = [];
    while (!empty($queue)) {
        $node = array_shift($queue);
        $result[] = $node;
        foreach ($graph[$node] as $neighbor) {
            if (--$inDegree[$neighbor] === 0) $queue[] = $neighbor;
        }
    }
    return count($result) === $numCourses ? $result : [];
}

```

Python

```

from collections import deque
def find_order(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    queue = deque(i for i in range(num_courses) if in_degree[i] == 0)
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return result if len(result) == num_courses else []

```

Go

```
func findOrder(numCourses int, prerequisites [][]int) []int {
    graph := make([][]int, numCourses)
    inDegree := make([]int, numCourses)
    for _, p := range prerequisites {
        graph[p[1]] = append(graph[p[1]], p[0])
        inDegree[p[0]]++
    }
    queue := []int{}
    for i := 0; i < numCourses; i++ {
        if inDegree[i] == 0 {
            queue = append(queue, i)
        }
    }
    result := []int{}
    for len(queue) > 0 {
        node := queue[0]
        queue = queue[1:]
        result = append(result, node)
        for _, neighbor := range graph[node] {
            inDegree[neighbor]--
            if inDegree[neighbor] == 0 {
                queue = append(queue, neighbor)
            }
        }
    }
    if len(result) == numCourses {
        return result
    }
    return []int{}
}
```

C++

```
vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses);
    vector<int> inDegree(numCourses, 0);
    for (auto& p : prerequisites) {
        graph[p[1]].push_back(p[0]);
        inDegree[p[0]]++;
    }
    queue<int> q;
    for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) q.push(i);
    vector<int> result;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        result.push_back(node);
        for (int neighbor : graph[node]) {
            if (--inDegree[neighbor] == 0) q.push(neighbor);
        }
    }
    return (int)result.size() == numCourses ? result : vector<int>{};
}
```

SECTION 18 — Dry Runs

Small Input

numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]] (course 1 needs 0; course 2 needs 0; course 3 needs both 1 and 2)

graph: 0→[1,2], 1→[3], 2→[3]

inDegree: 0:0, 1:1, 2:1, 3:2

queue=[0]

process 0: result=[0]; decrement 1→0, 2→0 → enqueue both → queue=[1,2]

process 1: result=[0,1]; decrement 3→1 → queue=[2]

process 2: result=[0,1,2]; decrement 3→0 → enqueue → queue=[3]

process 3: result=[0,1,2,3]; no outgoing edges → queue=[]

result.length=4=numCourses → valid order: [0,1,2,3]

Large Input (Conceptual)

For 10^5 courses and 10^5 prerequisite pairs, Kahn's algorithm processes each course once and each prerequisite edge once — $O(2 \times 10^5)$ total operations, confirming $O(V+E)$ regardless of the dependency structure's complexity.

Corner Case

Cyclic input: prerequisites=[[0,1],[1,0]], numCourses=2: inDegree: 0:1, 1:1 → queue starts empty (no in-degree-0 nodes) → loop never executes → result.length=0 numCourses=2 → correctly detected as impossible (cycle exists).

SECTION 19 — Advanced Concepts

- **Longest path in a DAG via topological order:** process nodes in topological order, maintaining a dist[] array where $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight}(u,v))$ for every edge $u \rightarrow v$ processed — since topological order guarantees u is finalized before v is processed, this DP relaxation is correct in a single $O(V+E)$ pass, a technique directly useful for critical-path scheduling problems.
- **Lexicographically smallest topological order:** replace Kahn's algorithm's plain FIFO queue with a min-heap, always processing the smallest available in-degree-0 node next — this guarantees the lexicographically smallest valid ordering among all valid orderings.
- **Alien Dictionary-style order inference:** when given a list of words assumed to be sorted according to some unknown alphabet order, compare adjacent word pairs to extract pairwise character-ordering constraints (building a directed graph edge for each inferred "this letter comes before that letter"), then apply standard topological sort to recover the full alphabet order (or detect that the given word list is contradictory/invalid).
- **Reverse-graph topological reasoning (Find Eventual Safe States):** some problems become topological-sort-friendly only after reversing all edges — the DAG of "who depends on me" versus "who do I depend on" — recognizing when to reverse edges is a subtle but powerful technique.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Topological Sort as the fundamental algorithm underlying **any dependency-resolution system** — build systems, package managers, CI/CD pipelines, and spreadsheet recalculation engines all reduce to this exact problem: “given a set of ‘must happen before’ constraints, find a valid execution order, or detect that no such order exists due to circular dependencies.” They’re also fluent in the natural extension of layering dynamic programming on top of the topological order to solve “longest/shortest path in a DAG” and “critical path” scheduling problems, a common and expected follow-up at the Senior/Staff level. Interviewers evaluate whether a candidate recognizes disguised topological-sort problems (like Alien Dictionary, which doesn’t mention graphs at all) and correctly validates completeness (cycle detection) rather than assuming any produced ordering is automatically valid.

SECTION 21 — Cheat Sheet

PATTERN: Topological Sort

RECOGNIZE: "prerequisites," "course schedule," "build order," "must happen before," directed dependency c

TEMPLATE (Kahn's Algorithm):

```
inDegree[v] computed for all v
queue = all nodes with inDegree == 0
result = []
while queue: node = dequeue(); result.append(node)
    for neighbor in adj[node]:
        inDegree[neighbor] -= 1
        if inDegree[neighbor] == 0: enqueue(neighbor)
if len(result) != n: CYCLE DETECTED (no valid order)
```

COMPLEXITY: $O(V + E)$ time and space

KEY PROOF: a DAG always has 1 in-degree-0 node available at every stage; incomplete processing = a cycle c

WATCH FOR: validating result completeness (cycle check), correct edge direction, tie-breaking requires min

DOESN'T APPLY WHEN: undirected graphs, graph has cycles (detect and report, don't force an order), need c
layer DP on top

SECTION 22 — Revision Notes (5-Minute Review)

- Topological Sort orders a DAG so every edge $u \rightarrow v$ has u before v .
 - Kahn’s algorithm (BFS): repeatedly process in-degree-0 nodes; incomplete processing = cycle detected.
 - DFS-based: postorder traversal, then reverse; needs separate 3-color cycle detection.
 - Multiple valid orders can exist; use a min-heap instead of a plain queue for lexicographically smallest.
 - Layer DP on top of topological order to solve longest/shortest path in a DAG.
 - “Alien Dictionary” style problems are disguised topological sort — infer edges from pairwise constraints first.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic cycle detection and ordering	Course Schedule (207), Course Schedule II (210)
Intermediate	Order inference, level-based scheduling	Alien Dictionary (269), Parallel Courses (1136)
Advanced	DP layered on topological order	Longest Path With Different Adjacent Characters (2246), Course Schedule IV (1462)
Expert	Multi-constraint, hierarchical topological sort	Sort Items by Groups Respecting Dependencies (1203), Parallel Courses III (2050), Build a Matrix With Conditions (2392)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Zero In, Go Next” (ZIGN) — nodes with Zero In-degree Go Next.
- **Visualization: Getting dressed** — socks before shoes, underwear before pants, but flexible ordering where no dependency conflicts.
- **Recognition shortcut:** “Prerequisites/dependencies/must happen before” + directed relationships → Topological Sort; incomplete processing = cycle exists.

SECTION 25 — Final Summary

Topological Sort produces a valid dependency-respecting linear ordering of a DAG by repeatedly processing nodes with no remaining unmet prerequisites (Kahn’s algorithm) or by reversing a postorder DFS traversal — both achieving $O(V+E)$. The single most important thing to remember forever: **a valid topological order only exists if the graph has no cycles, and Kahn’s algorithm’s incomplete processing (fewer nodes in the result than total nodes) is exactly the signal that a cycle exists — always validate this explicitly rather than assuming any produced sequence is automatically a correct answer.** Many seemingly unrelated problems (Alien Dictionary, critical-path scheduling) are topological sort in disguise, and layering a DP relaxation step on top of the topological order is the standard technique for solving longest/shortest-path problems in a DAG.